Report on Mini Project

Subject: Natural Language Processing (NLP)

AY: 2026–27

Transformer-Based Sentiment Classification Using Fine-Tuned DistilBERT

Sarthak Agrawal: 2303002

Ahmed Ansari: 2303009

Sunil Bhagat: 2303020

Yash Bhardwaj: 2303021

Guided by

Prof. Jayant Gadge

Abstract

This project implements binary sentiment analysis for movie reviews using a fine-tuned DistilBERT sequence-classification model. The IMDb dataset provides review text and sentiment labels, where 0 represents negative sentiment and 1 represents positive sentiment. A subset of 4,000 training reviews was used for the initial fine-tuning run. The model was trained for one epoch on a Google Colab Tesla T4 GPU. Evaluation on the full held-out IMDb test split of 25,000 reviews produced 88.13% accuracy. The project also demonstrates five NLP experiments: sentence boundary identification, tokenization, lemmatization, POS tagging, and WordNet-based semantic analysis.

1. Introduction

Sentiment analysis identifies the polarity expressed in text and is useful for analyzing reviews, feedback, and opinions. This project classifies movie reviews as positive or negative using a pretrained transformer fine-tuned on labeled examples.

2. Problem Statement

Given a movie review in natural language, predict whether the expressed sentiment is positive or negative, using labeled examples for training and a separate test set for evaluation.

3. Objectives

- Demonstrate five NLP preprocessing and linguistic-analysis experiments.
- Explore a labeled dataset of movie reviews.
- Convert reviews into numerical inputs using the DistilBERT tokenizer.
- Fine-tune a pretrained transformer for binary sentiment classification.
- Evaluate using accuracy, precision, recall, F1-score, and a confusion matrix.
- Test the model on new reviews and document its limitations.

4. Dataset

The project uses the IMDb dataset hosted on Hugging Face as stanfordnlp/imdb. The supervised training and test splits each contain 25,000 reviews. The original fields are text and label; label 0 denotes negative sentiment and label 1 denotes positive sentiment. For fine-tuning, 4,000 examples were selected from the training split. A 1,000-review subset was used for evaluation during training. Final evaluation used the full 25,000-review test split.

Field	Description
text	Original movie review.
label	Sentiment class: 0 = negative, 1 = positive.
input_ids	Token IDs generated by the DistilBERT tokenizer.
attention_mask	1 for actual tokens and 0 for padding positions.
token_type_ids	Returned by tokenizer in this run, then removed because not required by this DistilBERT model.

5. NLP Experiments

The five experiments were demonstrated separately in notebook cells using NLTK and WordNet. They demonstrate classical NLP analysis; their outputs were not all fed into DistilBERT training because the transformer uses its own tokenizer.

5.1 Sentence Boundary Identification

NLTK sent_tokenize splits text into individual sentences. In the sample, the paragraph was split into three sentences.

5.2 Tokenization

NLTK word_tokenize separates text into word and punctuation tokens. DistilBERT uses a separate subword tokenizer for model input.

5.3 Lemmatization

WordNetLemmatizer maps tokens to dictionary base forms. The demonstrated default noun mode changes “movies” to “movie”; other forms may need POS-aware lemmatization.

5.4 Syntactic Analysis: POS Tagging

NLTK pos_tag assigns grammatical tags such as DT, NN, VBD, and JJ to tokens.

5.5 Semantic Analysis Using WordNet

WordNet returns synsets and definitions for a word such as “fantastic,” demonstrating lexical meaning and semantic relationships.

Sentence boundary identification

```
from nltk.tokenize import sent_tokenize

sample_text = (
    "The movie was fantastic. The acting was brilliant! "
    "However, the ending was disappointing."
)

sentences = sent_tokenize(sample_text)

print("Original text:", sample_text)
print("\nDetected sentences:")
for i, sentence in enumerate(sentences, start=1):
    print(f"{i}. {sentence}")
```

```
Original text: The movie was fantastic. The acting was brilliant! However, the ending was disappointing.

Detected sentences:
1. The movie was fantastic.
2. The acting was brilliant!
3. However, the ending was disappointing.
```

Figure 1. Sentence boundary identification: code and output.

Tokenization

```
from nltk.tokenize import word_tokenize

tokens = word_tokenize(sample_text)

print("Original text:", sample_text)
print("\nTokens:")
print(tokens)
```

```
Original text: The movie was fantastic. The acting was brilliant! However, the ending was disappointing.

Tokens:
['The', 'movie', 'was', 'fantastic', '.', 'The', 'acting', 'was', 'brilliant', '!', 'However', ',', 'the', 'ending', 'was', 'disappointing', '.']
```

Figure 2. Tokenization using NLTK: code and output.

Lemmatization

```
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet

lemmatizer = WordNetLemmatizer()

words = ["movies", "acting", "was", "better", "running"]

print("Original -> Lemmatized")
for word in words:
    print(f"{word} -> {lemmatizer.lemmatize(word)}")
```

```
Original -> Lemmatized
movies -> movie
acting -> acting
was -> wa
better -> better
running -> running
```

Figure 3. Lemmatization using WordNetLemmatizer: code and output.

Syntactic analysis: POS tagging

```
pos_tags = nltk.pos_tag(word_tokenize(sample_text))

print("Word -> POS tag")
for word, tag in pos_tags:
    print(f"{word:15} -> {tag}")
```

```
Word -> POS tag
The             -> DT
movie           -> NN
was             -> VBD
fantastic       -> JJ
.               -> .
The             -> DT
acting          -> NN
was             -> VBD
brilliant       -> JJ
!               -> .
However         -> RB
,               -> ,
the             -> DT
ending          -> VBG
was             -> VBD
disappointing   -> VBG
```

Figure 4. POS tagging: grammatical tags assigned to tokens.

Semantic analysis using WordNet

```
from nltk.corpus import wordnet as wn

word = "fantastic"
synsets = wn.synsets(word)

print("Word:", word)
print("\nWordNet meanings:")
for synset in synsets[:5]:
    print("Synset:", synset.name())
    print("Definition:", synset.definition())
    print()
```

```
Word: fantastic

WordNet meanings:
Synset: antic.s.01
Definition: ludicrously odd

Synset: fantastic.s.02
Definition: extraordinarily good or great ; used especially as intensifiers

Synset: fantastic.s.03
Definition: fanciful and unrealistic; foolish
```

Figure 5. WordNet semantic analysis: synsets and definitions.

6. Model and Fine-Tuning Methodology

The model used was distilbert/distilbert-base-uncased with AutoModelForSequenceClassification and num_labels=2. DistilBERT provides contextual text representations, while a classification head produces scores for negative and positive sentiment.

6.1 Tokenization and Input Preparation

Reviews were mapped through the DistilBERT tokenizer to generate input_ids and attention_mask. The original text and token_type_ids were removed from the final training dataset, while input_ids, attention_mask, and label were retained. Dynamic padding was used to make sequences in each batch the same length.

6.2 Fine-Tuning Configuration

Parameter	Value
Pretrained model	distilbert/distilbert-base-uncased
Task	Binary sequence classification
Training subset	4,000 reviews
Epochs	1
Training batch size	16
Evaluation batch size	32
Learning rate	2×10^{-5}
Weight decay	0.01
Runtime	Google Colab with Tesla T4 GPU

During training, the model generated predictions, calculated loss against correct labels, propagated gradients backward, and updated trainable weights using an optimizer. This is actual fine-tuning rather than inference with an unchanged pretrained model.

6.3 Fine-Tuning Workflow

IMDb labeled reviews

↓

Select training subset and held-out test data

↓

DistilBERT tokenizer → input_ids and attention_mask

↓

Pretrained DistilBERT + two-class classification head

↓

Fine-tuning: loss, backpropagation, optimizer weight updates

↓

Fine-tuned model

↓

Evaluation on full 25,000-review test split

↓

Accuracy, precision, recall, F1-score, confusion matrix

7. Evaluation Metrics

Accuracy is the proportion of all predictions that are correct. Precision measures the correctness of predictions for a class. Recall measures the proportion of actual examples of a class that are correctly identified. F1-score is the harmonic mean of precision and recall. A confusion matrix displays true negatives, false positives, false negatives, and true positives.

8. Results and Discussion

Metric	**Negative**	**Positive**
Precision	0.8812	0.8814
Recall	0.8814	0.8812
F1-score	0.8813	0.8813

Overall metric	**Result**
Accuracy	88.13%
Macro F1-score	0.8813
Weighted F1-score	0.8813

Test samples	25,000

The model achieved 88.13% accuracy on the full test split. Scores for the negative and positive classes were very similar, indicating balanced performance on this dataset. Errors can still occur for sarcastic, ambiguous, or mixed-opinion reviews. The 1,000-review evaluation subset was used during training for evaluation; the full test split was separate from the 4,000 training examples and is the final reported test evaluation, provided it was not used to choose model settings.

Confusion Matrix Analysis



Figure 6: Confusion Matrix of Fine-Tuned DistilBERT on the IMDb Test Set.

The confusion matrix evaluates the fine-tuned DistilBERT model on 25,000 IMDb test reviews. It shows 11,018 true negatives (negative reviews correctly classified as negative), 1,482 false positives (negative reviews incorrectly classified as positive), 1,485 false negatives (positive reviews incorrectly classified as negative), and 11,015 true positives (positive reviews correctly classified as positive). The model correctly classified 22,033 out of 25,000 reviews, achieving 88.13% accuracy. The similar numbers of false positives and false negatives indicate comparable error counts for the two sentiment classes.

9. Example Predictions

The notebook's predict_sentiment function accepts a review, tokenizes it, runs the fine-tuned model, and returns a sentiment label and predicted class probability. Insert a screenshot of the actual notebook output here. Confidence is a model probability, not a guarantee of correctness.

Test review	**Actual model output**
it has many interesting scenes	'sentiment': 'Positive', 'confidence': 87.55
it was a total waste of time	'sentiment': 'Negative', 'confidence': 84.38

10. Libraries and Tools

Tool	Purpose
Python	Programming language.
Google Colab	Notebook runtime and GPU access.
PyTorch	Model tensor operations and training.
Hugging Face Transformers	DistilBERT tokenizer, model, and Trainer.
Hugging Face Datasets	IMDb dataset loading.
NLTK	Sentence splitting, tokenization, POS tagging, and lemmatization.
WordNet	Lexical-semantic analysis.
Scikit-learn	Classification metrics and confusion matrix.
Matplotlib	Evaluation visualization.

11. Limitations

- The model was fine-tuned for one epoch on a subset of the training split.
- The task predicts only positive or negative sentiment and does not explicitly model neutral sentiment.
- Sarcasm, negation, and mixed opinions can be difficult to classify.
- Softmax confidence is not necessarily calibrated.
- The five classical NLP experiments are demonstrated separately and are not all used as direct model inputs.
- BERTScore was not calculated. It is primarily used to compare generated text and is not a standard metric for binary classification; confirm the instructor's interpretation of this requirement.

12. Future Scope

- Compare DistilBERT with a TF-IDF plus Logistic Regression baseline.
- Experiment with additional epochs and learning rates using a validation set.
- Inspect misclassified reviews to analyze sarcasm and ambiguity.
- Explore a three-class sentiment dataset if neutral sentiment is required.
- Create a UI only if required by the project guidelines.

13. Conclusion

This project demonstrates binary sentiment classification using a fine-tuned DistilBERT model. It covers dataset preparation, transformer tokenization, actual fine-tuning, evaluation, and predictions on new reviews. The model achieved 88.13% accuracy on the full 25,000-review IMDb test split. Five NLP techniques were also demonstrated: sentence boundary identification, tokenization, lemmatization, POS tagging, and WordNet semantic analysis.

14. References

- IMDb dataset: https://huggingface.co/datasets/stanfordnlp/imdb
- DistilBERT model: https://huggingface.co/distilbert/distilbert-base-uncased
- Hugging Face Transformers: https://huggingface.co/docs/transformers
- NLTK: https://www.nltk.org/
- Scikit-learn model evaluation: https://scikit-learn.org/stable/modules/model_evaluation.html

Output:

```
))
```

```
{'review': 'This movie was fantastic. The acting was brilliant and I loved it!', 'sentiment': 'Positive', 'confidence': 94.87}
{'review': 'The movie was boring, badly written, and a complete waste of time.', 'sentiment': 'Negative', 'confidence': 94.19}
```

```
print(predict_sentiment("it was an interesting movie"))
```

```
{'review': 'it was an interesting movie', 'sentiment': 'Positive', 'confidence': 90.28}
```